

Роль выявления требований и определения приоритетов в Оптимизация качества в Scrum Agile-разработке

Анис Рида Асгар
Кафедра программной инженерии
Университет Бахрия, Исламабад, Пакистан

Шахид Назир Бхатти
Старший доцент,
Кафедра программной инженерии,
Университет Бахрия, Исламабад, Пакистан

Атика Табассум
Кафедра программной инженерии
Университет Бахрия, Исламабад, Пакистан

Зайнаб Султан, Рабия Аббас
Кафедра программной инженерии,
Университет Бахрия, Исламабад, Пакистан

Аннотация. Управление требованиями — один из наиболее распространённых аспектов традиционной разработки программного обеспечения. Требования возникают на протяжении всего процесса разработки программного обеспечения и, следовательно, должны быть реализованы посредством надлежащего взаимодействия и интеграции между заинтересованными сторонами, разработчиками и документацией. Гибкая методология — это инновационный и итеративный процесс, поддерживающий меняющиеся требования и помогающий учитывать изменения на протяжении всего процесса разработки.

Требования выявляются в начале каждого процесса разработки программного обеспечения и проекта (продукта), после чего им присваивается приоритет в соответствии с их важностью для рынка и самого продукта. Одним из наиболее важных и влиятельных этапов при создании программного продукта является приоритизация требований. Приоритизация требований помогает команде разработчиков понять наличие и важность конкретного требования, важность его использования и срочность вывода на рынок. Существует множество методов приоритизации требований со своими достоинствами и недостатками. Однако многие из них не учитывают все факторы, которые необходимо учитывать при приоритизации требований, такие как стоимость, ценность, риск, время вывода на рынок, количество требований и влияние нефункциональных требований на функциональные.

Существует несколько методологий приоритизации требований, которые помогают в принятии решений, но, что важно, во многих из них отсутствуют учет важных факторов, которые оказывают значительное влияние на приоритизацию требований. Была предложена методология приоритизации требований, основанная на нескольких факторах, таких как время вывода на рынок, стоимость, риск и т. д. Ожидается, что предлагаемая модель устранил этот недостаток. В спринтах требования будут приоритизироваться как на основе влияющих факторов, таких как стоимость, ценность, риск, время вывода на рынок и т. д., так и посредством влияния нефункциональных требований на функциональные. Это улучшит общее качество программного продукта, когда он будет включен в процесс разработки Scrum. Требования будут приоритизироваться не только на основе спринтов, человеческого решения, но и путем критического анализа факторов (подхарактеристик), которые могут привести к неоднократному успеху/неуспеху продукта, тем самым обеспечивая согласованность правильных требований и, следовательно, правильно приоритизированные требования будут выбираться для конкретного спринта за раз.

Ключевые слова — гибкая разработка программного обеспечения (ASE); гибкое программное обеспечение Разработка (ASD); Процесс разработки программного обеспечения Scrum; SCRUM; Владелец продукта (PO)

I. ВВЕДЕНИЕ

Традиционная разработка требований не поддерживает изменение требований и постоянное общение с заинтересованными сторонами, поэтому возникают проблемы, когда новые требования развиваются из-за изменения потребностей бизнеса и времени выхода на рынок [4] [3] [1]. Таким образом, рынок программного обеспечения движется в сторону гибкой разработки требований и быстрого изменения, поскольку она поддерживает изменение требований и быструю разработку процессов. Гибкие практики признаются и становятся популярными с каждым днем в области разработки требований. Один из самых популярных методов в семье agile, при котором программное обеспечение поставляется инкрементами, называемыми спринтами, известен как SCRUM [8] [6]. Спринт состоит из итерации продолжительностью 2-4 недели. Методология Scrum включает в себя совещание по планированию и ежедневное совещание Scrum, совещание по планированию проводится в начале каждого спринта. На этом совещании члены команды определяют количество требований, которые они могут обязаться выполнить, то есть они создают из этого бэклог спринта. Бэклог спринта содержит список всех задач, которые должны быть выполнены в течение определенного спринта. Ежедневные встречи Scrum длятся не более 15 минут, в течение которых владелец продукта (PO) получает непрерывную информацию о процессе разработки и может предоставить обратную связь о включаемых функциях.

Таким образом, если PO решает добавить новую функцию в спринт, он/она может обсудить это с командой разработчиков и сэкономить время, вместо того чтобы просматривать ее в конце и требовать внесения изменений в конце. Команда проводит обзор спринта в конце каждого спринта, демонстрируя новые функции и возможности владельцу продукта (PO) или другим заинтересованным сторонам, которые могут предоставить любую обратную связь, которая может быть полезна или полезна для следующего спринта. Этот цикл обратной связи приводит к изменениям в недавно предоставленной функциональности, с другой стороны, скорее всего, к пересмотру или добавлению новых требований в бэклог продукта. Еще одним видом деятельности в управлении проектами Scrum является ретроспектива спринта. В этой встрече участвуют Scrum-мастер, владелец продукта (PO) и команда разработки. Это возможность воспроизвести или проанализировать завершившийся спринт и определить новые пути для улучшения. Scrum состоит из трех артефактов: бэклогов спринта, бэклогов продукта и диаграмм выгорания. Бэклог продукта, расставленный владельцем продукта по приоритетам, представляет собой полный список функциональности (записанной в виде пользовательских историй), которая должна быть добавлена в продукт в конечном итоге. Он расставлен по приоритетам, чтобы команда могла

в первую очередь работайте над самыми важными, срочными и ценными функциями. С другой стороны, бэклог спринта — это список всех задач, которые команда обязана выполнить в течение спринта для предоставления требуемой функциональности. Оставшийся объем работы в спринте или релизе отображается в виде диаграмм «Burn down» . Это эффективный инструмент для оценки соответствия спринта или релиза графику и своевременного завершения всех запланированных работ. Традиционная разработка требований очень трудоёмка и требует быстрого процесса для своевременного удовлетворения потребностей рынка, поэтому современная индустрия программного обеспечения требует быстрых и итеративных процессов, таких как гибкая разработка, чтобы успевать за меняющимися требованиями и временем.

На успех или неудачу продукта влияет множество факторов, одним из которых является сбор и приоритизация требований [2]. Выявление требований и приоритизация являются одной из самых сложных задач во время разработки продукта, и вряд ли удастся записать все требования на ранней стадии, они постоянно меняются на протяжении всего процесса разработки и должны быть рассмотрены должным образом, чтобы соответствовать меняющимся потребностям рынка и времени. Поскольку Scrum является гибкой методологией, поэтому он позволяет инженерам справляться с изменяющимися требованиями по мере их развития; однако по-прежнему сложно понять, какие предварительные условия достаточно важны, чтобы иметь высокую потребность и быть включенными в ранние спринты. Организация требований в приоритетные требования помогает команде проекта понять, какие требования являются наиболее важными и наиболее срочными для реализации и выполнения. Приоритизация также является полезной деятельностью для принятия решений на других этапах разработки программного обеспечения.

Поэтому в процессы Scrum должна быть включена хорошо управляемая методика приоритизации требований, которая повышает их качество.

Предложен метод приоритизации требований, основанный на нескольких факторах, таких как время вывода на рынок, стоимость, риск и т. д. Это повысит качество программного продукта при его включении в процесс разработки Scrum. Ожидается, что предлагаемая модель позволит преодолеть нехватку качества, характерную для предыдущих моделей. Приоритет требований будет определяться как на основе таких влияющих факторов, как стоимость, ценность, риск, время вывода на рынок и т. д., так и с учётом влияния нефункциональных требований на функциональные. Приоритет требований будет определяться не только на основе человеческого решения, но и на основе критического анализа факторов, которые могут привести к многократным провалам/успехам продукта, обеспечивая тем самым единообразие правильных требований для каждого спринта.

II. МАТЕРИАЛЫ И МЕТОДЫ

A. Гибкая разработка требований

В этой работе [1] автор представил десятилетний прогресс исследований гибких методологий и предложил некоторые будущие направления исследований, в которых исследователи гибких методологий могут придерживаться теоретического или гипотетического подхода. Для получения достоверной информации о прогрессе гибких методологий была использована методология, основанная на литературе. Важно помнить, что создание и развитие новых областей как научной дисциплины возможно только при наличии возможностей для создания прочной теоретической системы для проведения исследований в области гибкой разработки. Поэтому в будущем при изучении областей, в которых гибкая разработка является конкурентоспособной, исследователям, работающим в этой области, необходимо придерживаться более теоретического подхода.

Мин Хо и др. [3] предположили, что гибкие методы могут гарантировать качество, даже если они быстрее и должны учитывать меняющиеся требования. Автор, по сути, представил сравнение каскадной и гибкой моделей и результаты. Гибкие методы содержат ряд практик, обладающих возможностями обеспечения качества, поэтому с помощью гибких методов можно более эффективно достичь этого качества. Однако при документировании гибкой разработки программного обеспечения необходимо учитывать, что в сложных процессах разработки ПО недостаточное количество документации может привести к некоторым проблемам.

Лань Цао и др. [4] представили эмпирическое исследование гибких методов разработки решений (RE). Это исследование показывает, что разница между гибким RE и традиционным RE заключается в итеративном подходе к поиску решений. Разработка чёткой и полной спецификации требований в рамках гибкой разработки невозможна. В связи с такими важными различиями в практику, описанную в данной статье, вошёл новый набор гибких методов разработки решений (RE). Участники исследования признали, что важнейшей практикой в RE является эффективная коммуникация между разработчиками и заказчиками.

Многие участники подчеркнули, что эффективность этой практики во многом зависит от полноценного взаимодействия и коммуникации между заказчиками и разработчиками. Отсутствие качественного взаимодействия в любом проекте приводит к таким рискам, как неполные, неэффективно разработанные или неверные требования.

В этой работе Пекка и др. [6] предположили, что существуют различные методы гибких процессов, требующие эмпирических доказательств. Авторы акцентировали внимание на качестве методологии, а не на количестве. Этот подход был выбран для сравнительного анализа этих процессов. В аналитические линзы включены пять перспектив. SDLC включает в себя абстрактные принципы в сравнении с конкретными рекомендациями, эмпирическими доказательствами, управлением проектами и универсально предопределёнными в сравнении с подходящими для ситуации. На основе этих пяти перспектив предлагаются новые направления, ориентированные на качество, а не на количество методов.

Амин и др. [7] предположили, что некоторые аспекты RE должны учитываться в гибких методологиях, если основное внимание уделяется качеству. К числу важных аспектов RE, которым в Agile уделяется недостаточно внимания, относятся анализ (верификация и валидация), нефункциональные требования и управление изменениями. Автор предположил, что эти практики RE могут быть внедрены в Agile и позволят достичь высокого качества. Такие практики RE, как простота, непрерывная валидация, короткие релизы и частый рефакторинг, могут быть реализованы с точки зрения основных идей Agile.

Дипти Мишра и др. [8] предположили, что гибкий процесс может быть полезен при разработке сложных программных проектов. Автор подкрепил свой аргумент примерами из практики. Среднее предприятие (МСП), применявшее гибкие методы, добилось множества успешных результатов. Начало проекта с применением гибких методов и последующее достижение оптимальных результатов путём адаптации гибких методов к видению и выгодам проекта. Причина успеха управления цепочками поставок. Архитектурное проектирование этого масштабного сложного проекта было подкреплено официальной документацией. Эта проектная документация сыграла важную роль в успешном завершении проекта.

Франек и др. [11] предложили различные способы применения методов RE, которые могут дать преимущества гибкой разработке программного обеспечения. Также обсуждаются некоторые общие и различные черты и атрибуты

традиционных и гибких подходов.

Гибкие подходы, такие как XP, предполагают обратную связь от команд разработчиков и клиентов, коммуникацию и простоту. Аналогично, процесс разработки решений (RE) также включает в себя диктат, анализ и валидацию. Однако в гибком процессе этапы не так четко разделены, как в процессе разработки решений (RE), и методы также могут различаться. В целом, оба подхода преследуют одни и те же цели. Основное различие заключается в документации, которая действительно важна для взаимодействия с заинтересованными сторонами.

В.Н. Витана [12] провел исследование с использованием качественных методов, чтобы выяснить, какие практики инженерии требований чаще всего используются в методологии SCRUM при разработке программного продукта за рубежом. Для сбора данных были опрошены представители различных должностей из девяти организаций. Было обнаружено, что такие практики RE, как вовлечение клиентов, создание прототипов, разработка на основе тестирования и взаимодействие, являются наименее практикуемыми видами деятельности по проектированию требований, хотя большинство членов команды успешно практиковали итеративное проектирование требований, личное общение, управление изменениями требований и приоритизацию требований практики SCRUM RE.

Б. Методы приоритизации требований

В этой работе Анна Перини и др. [14] предложили стратегию, называемую ранжированием на основе прецедентов (CBRank). Этот метод объединяет предпочтения заинтересованных сторон проекта с аппроксимацией порядка требований, вычисляемой с помощью методов машинного обучения. На моделируемых данных свойства CBRank проверяются, а затем сопоставляются с методом, называемым современным методом приоритезации, что позволяет получить эмпирические результаты.

Однако в процессе приоритезации CBRank имеются некоторые предположения, такие как произвольный выбор в качестве политики парной выборки и монотонность процесса выявления.

Для повышения эффективности методов комплексного анализа реальных ситуаций авторы намерены в будущем работать над более совершенными стратегиями немонотонной формальной логики и парной выборки.

Дэн Хао и др. [10] в этой статье представили стратегию, включающую в себя общую и дополнительную стратегии для унифицированной приоритезации тестовых случаев. Эти тактики приоритезируют тестовые случаи с учётом компонентов, защищённых в каждом тестовом случае, совокупного количества сегментов программы (или связанных с кодом) и количества других (ещё не охваченных) сегментов программы (или связанных с кодом) компонентов, охваченных в каждом тестовом случае, соответственно. Предлагаемый подход включает в себя базовую и расширенную модели, которые определяют спектр приоритезации тестовых случаев от чисто общей до чисто дополнительной, путём указания значения параметра, называемого значением α [10].

Рахул Такурта [15] предложил количественную структуру, которая определяет приоритет списка нефункциональных требований. Эта структура включает в себя участников бизнес-организации и проекта, обеспечивая измеримую основу для оценки уровня добавленной стоимости, учитываемого при выборе нового нефункционального требования к набору требований проекта. Однако данные, предоставленные участниками структуры, были субъективными, что может привести к неоптимальным результатам. Кроме того, поскольку процесс оценки требований

В процессе оценки участвуют заинтересованные стороны как из бизнес-структур, так и из проекта, что может привести к несовместимости интересов и приоритетов требований. Автор также обозначил направления дальнейшей работы, направленной на создание эвристики, ограничивающей число заинтересованных сторон, предпочтительных для процесса оценки.

Наила Шариф и др. [16] разработали новый метод приоритезации требований под названием FuzzyHCV, который представляет собой гибрид двух доменов (SE и Computational Intelligence). Он представляет собой слияние двух методологий: иерархического кумулятивного голосования (HCV) и нечеткой экспертной системы. В FuzzyHCV вместо одного четкого значения используется треугольное нечеткое число. Предложенный метод был применен к 3 тематическим исследованиям, и полученные результаты очень близки к результатам фактической приоритезации, использованной во всех трех тематических исследованиях. Установлено, что FuzzyHCV дает более точные результаты, чем HCV, путем сравнения их с фактическими результатами для выбранных наборов данных. Авторы намерены продолжить работу в этой области, используя fuzzyHCV для решения проблем других доменов, таких как проблемы принятия решений при отборе сотрудников, и путем включения fuzzyHCV в уже существующие методы принятия решений или приоритезации требований, чтобы в будущем делать менее рискованный выбор.

Нупул Кукреджа и др. [17] в своей работе предложили методологию приоритезации требований к системе и программному обеспечению. Эта методология представляет собой двухэтапный подход, основанный на теоретической модели принятия решений с использованием алгоритма приоритезации TOPSIS. В предлагаемом подходе [13] система изначально фрагментируется на высокоуровневые минимальные рыночные функции (MMF). Предлагаемая методология позволяет оценить влияние колебаний бизнес-приоритетов на отдельные требования без существенных накладных расходов. Эта

методология также позволяет заинтересованным сторонам проводить многочисленные анализы, которые также помогают точно оценить влияние меняющихся бизнес-приоритетов на отдельные требования.

Здесь авторы также представили отчет о валидации данной методологии, внедрённой в 24 проектных группах студентов, обучающихся по программе «Программная инженерия» в Университете Южной Калифорнии. Хотя этот подход имеет ряд недостатков, которые необходимо устранить в будущем, например, одним из недостатков TOPSIS является изменение рангов, то есть исходный порядок приоритезации требований может измениться при добавлении нерелевантных требований. Это ограничение не учитывалось при реализации подхода, поскольку команды были ориентированы на результат и поэтому воздерживались от добавления нерелевантных требований для приоритезации. Другой недостаток заключается в том, что упорядоченная приоритезация требований может...

неточно отражают ожидаемый ранжированный порядок требований

Чтобы преодолеть недостатки TOPSIS, вместо TOPSIS можно использовать несколько других алгоритмов приоритезации, например, стоимость задержки, простое аддитивное взвешивание или алгоритм Вейгера. Приоритезация. Также можно просто записывать пункты, чтобы исключить их. накладные расходы, связанные с неспособностью winbook регистрировать элементы.

III. СТРУКТУРА ПРИОРИТЕТИЗАЦИИ ТРЕБОВАНИЙ

Отсутствие или нечеткое определение требований к качеству может привести к провалу проекта или огромным потерям. Эффективное выявление требований к качеству — сложная задача, особенно в SCRUM.

Один человек, то есть владелец продукта (PO), должен составить список всех требований, которые должны быть включены в проект. Это может быть напряженной и сложной задачей. Поскольку требования к качеству определяют архитектуру систем с большим объемом программного обеспечения, они важнее функциональных требований. Таким образом, успех или неудача критически важных систем зависит от того, насколько хорошо разработаны и реализованы требования к качеству. Приоритизация требований также является сложной задачей при разработке программного продукта. Владельцы продукта обычно используют следующие методы приоритизации бэклога: анализ Кано, метод Москвы и метод относительного взвешивания (Карл Вигер) [3] [8].

IV. МЕТОДОЛОГИЯ

Предлагаемая модель основана на нескольких методах, используемых для приоритизации требований. Однако ожидается, что их сочетание даст лучшие результаты. Первым шагом в этой модели является кумулятивное голосование. При кумулятивном голосовании каждый стейкхолдер распределяет в общей сложности 100 баллов (\$, евро или монет) по требованиям, затем владелец продукта суммирует баллы и представляет полученный порядок требований. Хотя желаемые функции будут выбраны на этом этапе, существует вероятность, что выбранная функция не обеспечит таких преимуществ с точки зрения стоимости, времени или простоты, как это могло бы быть с другими функциями, выбранными на этом этапе. Вторым шагом является численное назначение требований; это наиболее распространенный метод приоритизации требований, основанный на группировке требований в различные приоритетные группы. Например, сгруппируйте требования, собранные на первых этапах, в различные группы в зависимости от их характера, такие как требования к риску, требования к ценности, требования к сложности и т. д. После этого требования будут сгруппированы на основе влияющих факторов, которые могут каким-либо образом повлиять на эти требования. Например, R1 и Rn — это требования к риску [11] (см. рис. 2 ниже), которые каким-либо образом противоречат другим требованиям, выбранным для реализации в спринте. На рис. 1 представлены этапы предлагаемой методологии.

Это может привести к сложностям в реализации всех этих требований, поэтому это следует учитывать при выборе и приоритизации требований для спринта. Далее группы будут ранжированы по наивысшим баллам (см. рис. 2). Группы с требованиями R1, R3, R4 в целом набирают больше баллов, чем остальные группы, поэтому они имеют более высокий приоритет. После этого следующим шагом является АНР, в котором приоритеты требований рассчитываются для оценки их относительной важности путем сравнения всех уникальных пар требований. Другими словами, сравнивающий вручную определяет, какое требование имеет большую значимость и в какой степени, используя шкалу от 1 до 9.[14] АНР обеспечивает лучшие результаты, чем любые другие протестированные методы, поскольку это методология шкалы отношений, а также включает проверку согласованности.

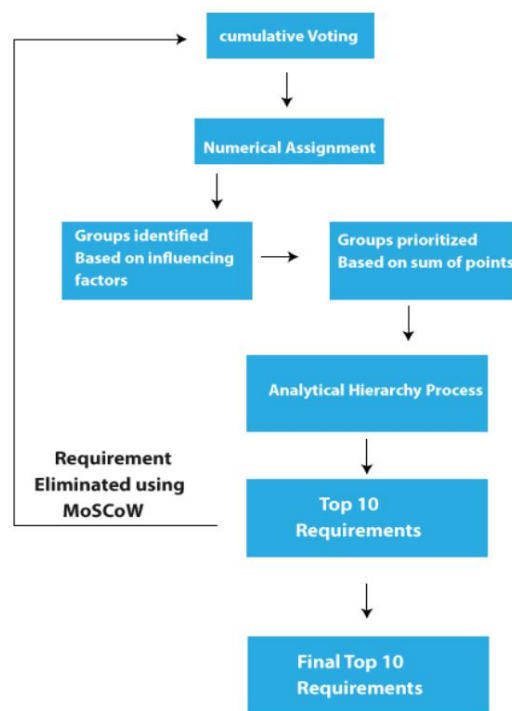


Рис. 1. Гибридная модель приоритизации требований

Шаги, предусмотренные АНР:

1) Создайте матрицу $v \times v$ (v представляет собой количество требований), затем требования вставляются в строки и столбцы матрицы.

2) Для каждой пары требований укажите их относительную степень важности (в точке пересечения строки X и столбца Y). В той же точке укажите обратные значения в транспонированных позициях (например, если ячейка $XY = 4$, то ячейка $YX = 1/4$).

3) Теперь вычислите собственные значения этой матрицы, чтобы определить относительный приоритет каждого требования. Конечный результат и будет относительным приоритетом требований.

Общее количество сравнений, требуемых АНР, составляет $v \times (v - 1)/2$. Избыточность возникает при парных сравнениях в АНР, поэтому АНР также вычисляет коэффициент согласованности для проверки точности сравнений [14].

На этом этапе, когда небольшое количество требований уже выбрано и сгруппировано, лучше всего применить АНР, поскольку группировка требований по их характеру и факторам влияния позволит легко сравнить требования с другими группами и выявить их относительную важность или противоречия между ними. Как команда Agile-разработки,

Поскольку у РО есть наилучшее представление о реализации таких требований, которые в некоторой степени противоречат друг другу, и/или о рисках или затратах, связанных с их реализацией, на данном этапе рекомендуется применять MoSCoW. MoSCoW основан на мнении людей, основанном на их опыте, желаниях и влияющих на них факторах, таких как рыночный спрос, стоимость, риск, время и ресурсы. Полученные в результате требования затем снова фильтруются с помощью MoSCoW. Ожидается, что это отфильтрует те требования, которые могли получить более высокие баллы в 100-долларовом тесте (кумулятивное голосование), но противоречат другим требованиям или могут быть менее выгодными для их реализации в этом спринте.

Новые требования из бэклога добавляются после того, как уже существующие требования были отфильтрованы. Если количество добавленных требований превышает 3 или 4, все шаги повторяются для этих требований. Если добавляется небольшое количество требований, следует применять только MoSCoW.

Подробная схема предлагаемой модели представлена ниже.

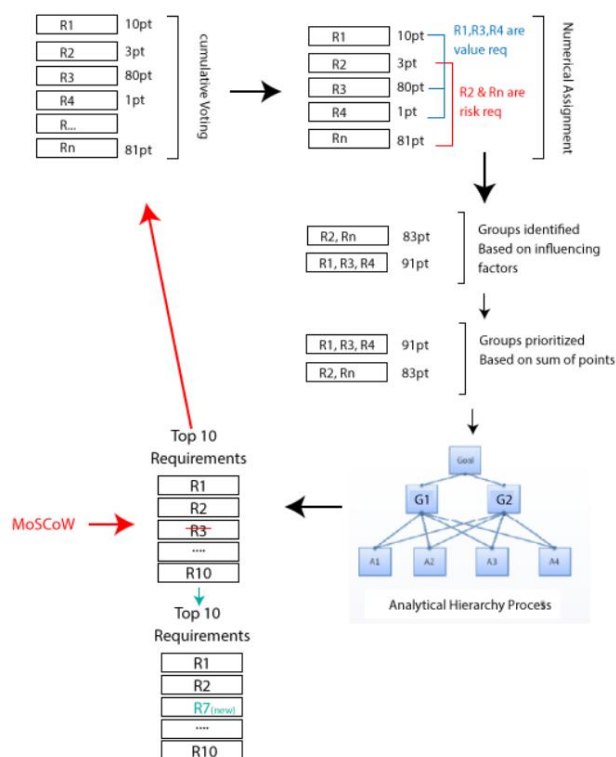


Рис. 2. Детальное представление предлагаемой модели

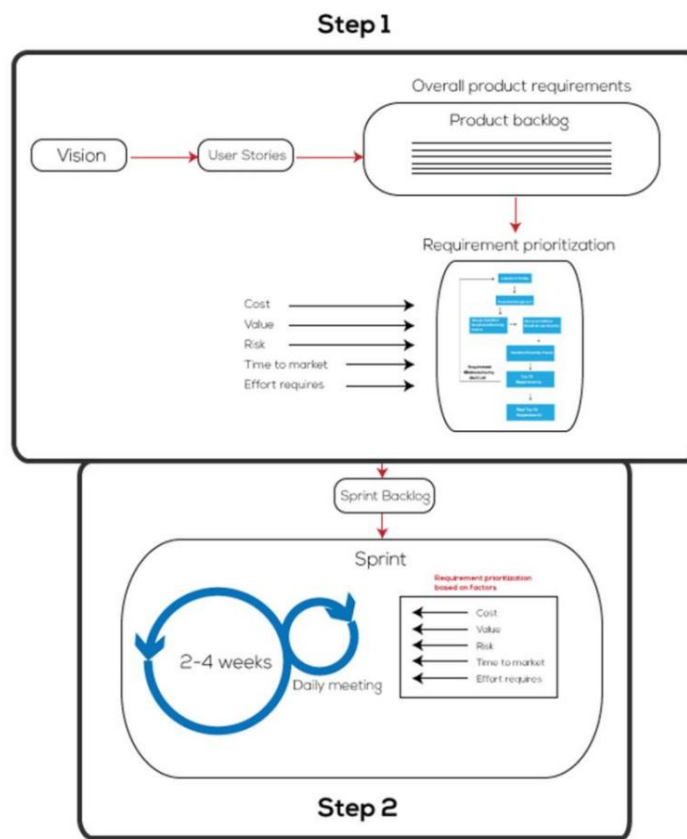


Рис. 3. Детальная реализация (стадирование) в скраме

Валидация предлагаемой модели проводится посредством детального моделирования с использованием программного обеспечения iThink (см. рис. 4 ниже). Ожидается, что модель повысит качество отбора и приоритизации требований в ходе спринтов при гибкой разработке SCRUM. Моделирование показывает, что по мере увеличения количества требований в процессе разработки (см. диаграмму 1) и добавления новых требований после отфильтровывания противоречащих другим, приоритет требований изменяется по мере добавления новых требований после применения выбранных методов (см. диаграмму 2). Результаты этих изменений в требованиях и приоритизации показывают, что качество отбора требований и приоритизации повышается (см. диаграмму 3) и, как ожидается, даст лучшие результаты при внедрении гибкой разработки SCRUM.

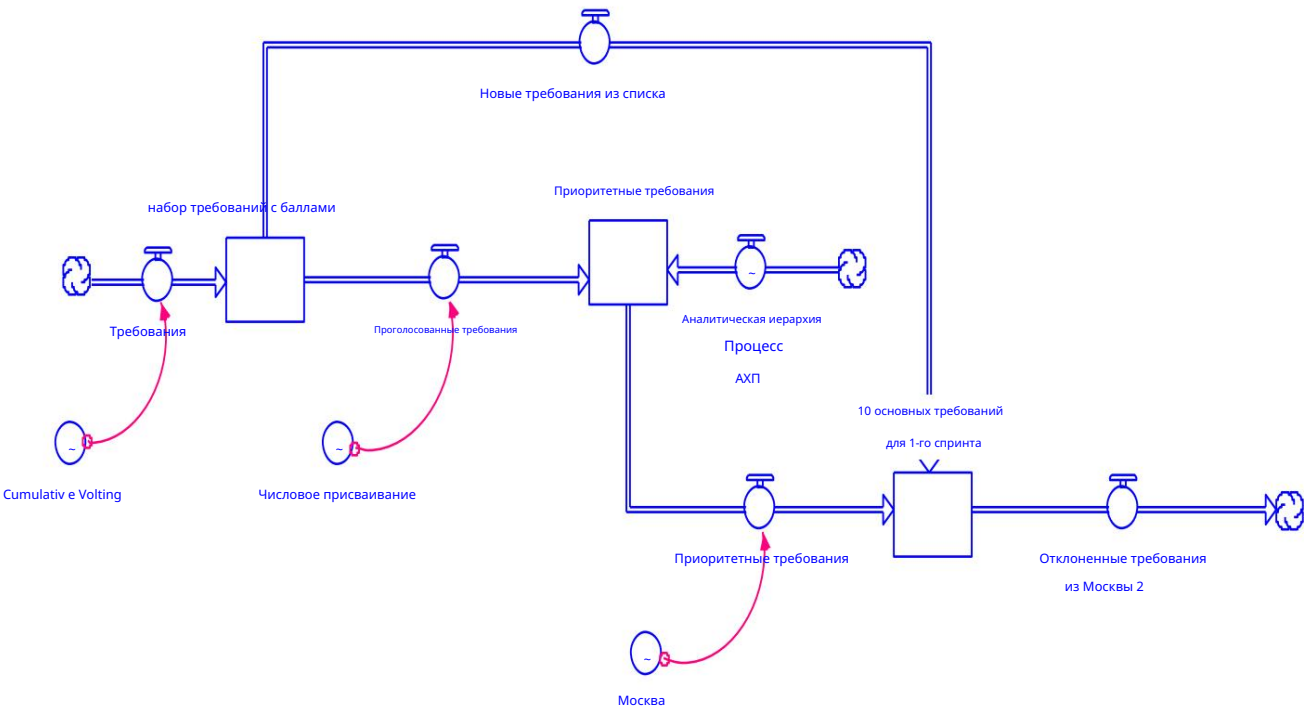


Рис. 4. Моделирование предлагаемой модели в iThink

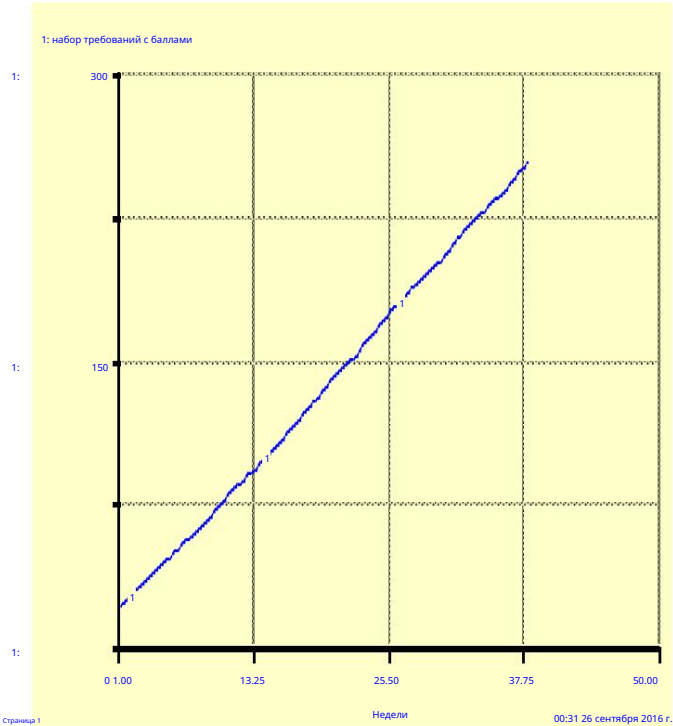


Рис. 5. Диаграмма, показывающая рост требований во время SDLC. Без

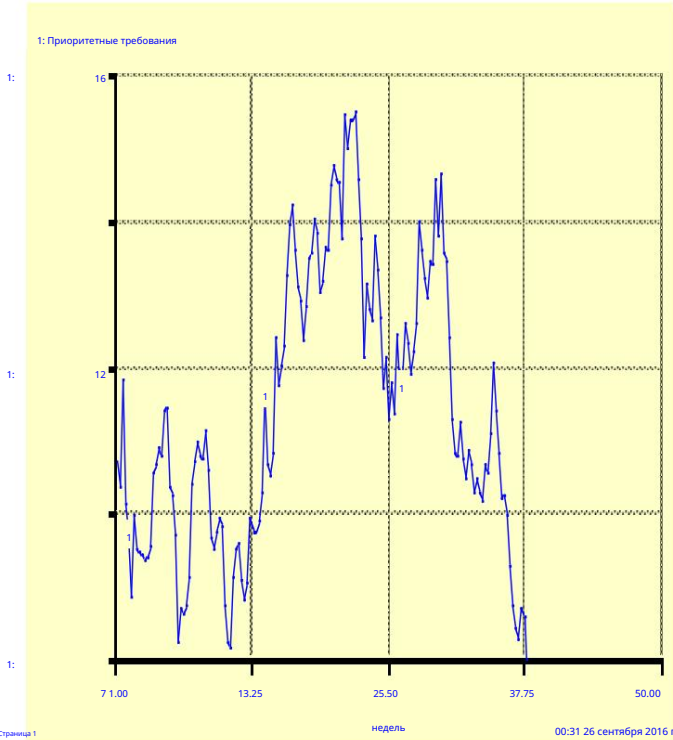


Рис. 6. Диаграмма, показывающая приоритет изменения требований по мере поступления новых требований.

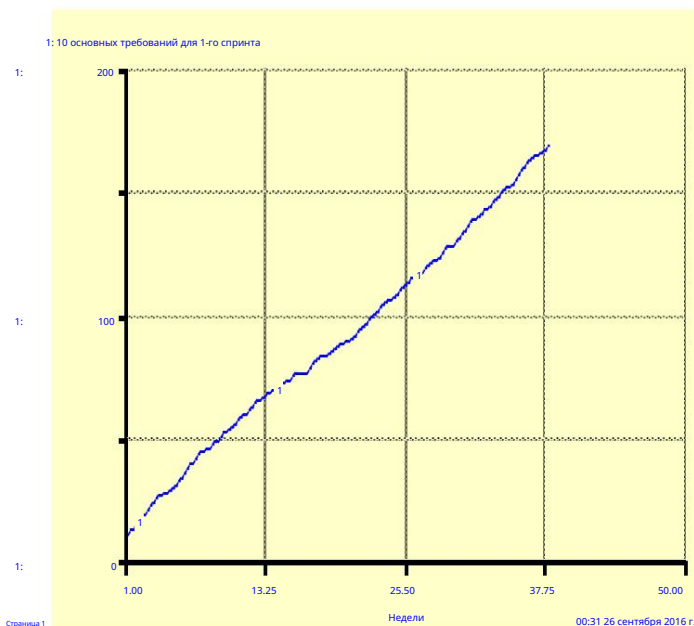


Рис. 7. Диаграмма показывает, что качество требований повышается при применении без к предлагаемой модели для особого набора требований

V. ЗАКЛЮЧЕНИЕ

Поскольку требования возникают на протяжении всего процесса разработки программного обеспечения, их необходимо приоритизировать и, следовательно, управлять ими с наивысшим приоритетом, особенно в сценарии гибкой разработки программного обеспечения. Как уже упоминалось и подчеркивалось в данной исследовательской работе, существует множество методов и методологий приоритизации требований, которые были предложены и применялись, но большинство из них не учитывают классические факторы (метрики), влияющие на общее качество разрабатываемого программного продукта, например, внешние метрики ISO (9126, 25000). В данной исследовательской работе была предложена методология, в которой мы учитывали упомянутые внешние метрики ISO/IEC (т. е. 25000, 9126), влияющие как на качество процесса, так и на качество продукта. Кроме того, как можно ясно видеть, эти упомянутые показатели (атрибуты) повышают качество требований, выбираемых для разработки продукта, учитывая все те аспекты, которые влияют на приоритизацию требований, особенно в случае ISO 25000. Предлагаемая здесь модель является гибридом других методов приоритизации требований, она обладает преимуществами всех положительных особенностей, уже имеющихся

те предписанные методы, как упоминалось, а также имеют проверку согласованности, которая гарантирует, что правильные требования выбираются в нужное время для спринта в рамках процесса в случае ASE (Scrum).

ССЫЛКИ

[1] Elsevier (2012) Десятилетие гибких методологий: к объяснению гибкой разработки программного обеспечения, Журнал систем и программного обеспечения

[2] Мохаммад Муким, д-р Мохаммад Ризван, «Валидация структуры выявления требований с использованием конечного автомата», Международная конференция IEEE по управлению, приборостроению, связи и вычислительным технологиям (ICCICST), стр. 1210–1216, 2014.

[3] Мин Хо, Джун Вернер, Лиминг Чжу, Мухаммед Али Бабар (2004) Качество программного обеспечения и гибкие методы, IEEE

[4] Лан Цао, Баласубраманиам Рамеш (2008) Требования Agile. Инженерная практика: эмпирическое исследование, IEEE.

[5] Шахид Назир, SEN-2005, Почему качество? Поддержка показателей качества программного обеспечения ISO 9126 (функциональность) с помощью UML Suite, Нью-Йорк, США. DOI=<http://dl.acm.org/citation.cfm?id=1050849.1050860>

[6] Пекка Абрахамссона, Юхани Варстаб, Микко Т. Сипоненб и Юсси Ронкайнен (2003) Новые направления в гибких методах: сравнительный анализ, IEEE.

[7] Армин Эберлейн, Хулио Сезар Сампайо ду Прадо Лейте (2002) Определение гибких требований: взгляд с точки зрения разработки требований, Труды Международного семинара по разработке требований.

[8] С. Н. Бхатти, Выведение сложности из качества системы с использованием UML. Заметки по программной инженерии ACM SIGSOFT 34(3): 1-7 (2009). DOI= <http://dl.acm.org/citation.cfm?id=1527202.1527207> [9] ДАН ХАО, ЛИНМИН ЧЖАН, ЛУ ЧЖАН, ГРЕГГ РОТЕРМЕЛЬ, ХУН МЭЙ, (2014) Унифицированный подход к приоритизации тестовых случаев, Труды ACM по программной инженерии и методологии, том 24, № 2, статья 10, дата публикации: декабрь 2014 г.

[10] Фрауке Паэтч, Фрауке Паэтч, доктор Франк Маурер (2003) Требования к проектированию и гибкой разработке программного обеспечения, IEEE

[11] VN Vithana (2015) Практики разработки требований Scrum и проблемы оффшорной разработки программного обеспечения, Международный журнал компьютерных приложений (0975 – 8887), том 116 – № 22, апрель 2015 г.

[12] Азар, Дж., Смит, Р.К., «Приоритизация ценностно-ориентированных требований в небольшой организации по разработке», IEEE Computer Society, 2007, стр. 32–37, 2007.

[13] Анна Перини, Анджело Сузи, Паоло Авесани (2013) Подход машинного обучения к приоритизации требований к программному обеспечению, IEEE TRANSACTIONS ON SOFTWARE ENGINEERING, TOM 39, № 4, АПРЕЛЬ 2013 Г.

[14] Рахул Такурта (2013) Структура определения приоритетов требований к качеству для включения в программный проект, Качество программного обеспечения Журнал (2013) 21:573–597

[15] Наила Шариф, Кашиф Зафар, Вакас Зьяд (2014) Оптимизация Приоритизация требований с использованием метода вычислительного интеллекта, Международная конференция по робототехнике и новым смежным технологиям 2014 года Технологии в инженерии (iCREATE) Исламабад, Пакистан, 22 апреля 24, 2014

[16] Нулуп Кукрея, Барри Бём (2013) Интеграция сотрудничества Процессы согласования требований и определения приоритетов: брак, заключенный на небесах. Материалы Международной конференции 2013 г. Программное обеспечение и системный процесс

[17] Рубайда Исмин, Алим Уль Гиас, Шах Мостафа Халед (2014) Частичное Метод ассимиляции заказов для определения приоритетов требований к программному обеспечению. 3-я МЕЖДУНАРОДНАЯ КОНФЕРЕНЦИЯ ПО ИНФОРМАТИКЕ, ЭЛЕКТРОНИКЕ И ЗРЕНИЕ 2014

[18] Шахид Н. Бхатти, Мария Усман, Амр А. Джади, 2015, Валидация по фреймворку выявления требований с помощью метрик. ACM SIGSOFT Software Engineering Notes 40(5): 17, США. DOI= <http://dl.acm.org/citation.cfm?id=2815021.2815031> [19] Дж. Карлссон и К. Райан. 1997, «Приоритизация требований с использованием подхода «стоимость-ценность», IEEE Software 14(5), стр. 67–74.

[20] Джон А. Макдермид, Справочник инженера-программиста, Баттерворт-Хайнеманн, 1991.